

پژوهشگر: محسن رجب پور رودمعهجی

فهرست مطالب

۴	۱ مقدمه
۵	۲ خروجی ۱
۵	۱ ایجاد جدول جلورانی و ساخت درخت جستجو
۵	۱-۱ ورود جدول جلورانی
۶	۲-۱ ایجاد درخت جستجو با گام‌های مختلف
۶	۳-۱ چاپ ساختار درخت
۱۳	۲ تولید آدرس‌های مقصد تصادفی و بررسی صحت عملکرد
۱۳	۱-۲ تولید آدرس‌های تصادفی
۱۳	۲-۲ بررسی صحت عملکرد جستجو
۱۳	۳-۲ نتایج بررسی
۱۴	۴-۲ تحلیل نتایج
۱۴	۳ نتیجه‌گیری
۱۵	۳ خروجی ۲
۱۵	۱ بررسی کارایی درخت‌های جستجو با گام‌های بلند ۱، ۲، ۴ و ۸
۱۵	۱-۱ ایجاد درخت‌ها
۱۵	۲-۱ تولید آدرس‌های مقصد
۱۵	۳-۱ الف) مقایسه حجم حافظه مصرفی
۱۶	۴-۱ ب) مقایسه زمان جستجو

۲	نتایج بررسی کارایی	۱۸
۳	نتیجه‌گیری	۱۸

فهرست تصاویر

۶	۱-۲ جستجو و میانگین زمان با گام ۱
۶	۲-۲ نتایج جستجو با گام ۱
۷	۳-۲ ساختار درخت با گام ۱
۸	۴-۲ جستجو و میانگین زمان با گام ۲
۸	۵-۲ نتایج جستجو با گام ۲
۹	۶-۲ ساختار درخت با گام ۲
۱۰	۷-۲ میانگین زمان جستجو به همراه بخشی از ساختار با گام ۴
۱۱	۸-۲ ساختار درخت با گام ۴
۱۲	۹-۲ ساختار درخت با گام ۴
۱۲	۱۰-۲ نتایج جستجو با گام ۴
۱۶	۱-۳ Stride1
۱۶	۲-۳ Stride2
۱۶	۳-۳ Stride4
۱۶	۴-۳ Stride8
۱۷	۵-۳ خروجی مقایسه زمان جستجو

فصل ۱

مقدمه

در شبکه‌های کامپیوتری و سیستم‌های مسیریابی، یکی از وظایف حیاتی، یافتن سریع و کارآمد مسیر مناسب برای ارسال بسته‌های داده به مقصد مورد نظر است. این فرآیند که به عنوان جستجوی آدرس‌های IP شناخته می‌شود، تأثیر مستقیمی بر کارایی و سرعت شبکه دارد. با افزایش حجم ترافیک شبکه و تعداد آدرس‌های موجود در جدول‌های مسیریابی، نیاز به الگوریتم‌ها و ساختار داده‌های بهینه برای جستجوی سریع و استفاده مؤثر از منابع سیستم احساس می‌شود.

درخت‌های جستجوی چند بیتی (Multibit Trie) از جمله ساختار داده‌هایی هستند که به دلیل قابلیت جستجوی سریع و مصرف حافظه مناسب، در پیاده‌سازی جدول‌های مسیریابی مورد استفاده قرار می‌گیرند. این ساختار با بهره‌گیری از پردازش چند بیت در هر مرحله (بلند گام)، عمق درخت و تعداد مراحل جستجو را کاهش می‌دهد، که منجر به بهبود زمان جستجو می‌شود.

در این پروژه، به بررسی و پیاده‌سازی الگوریتم جستجوی آدرس‌های IP بر اساس درخت جستجوی چند بیتی می‌پردازیم. هدف اصلی، آشنایی و درک بیشتر با این الگوریتم و ارزیابی کارایی آن با گام‌های بلند مختلف (۱، ۲، ۴ و ۸) است. برای این منظور، برنامه‌ای کامپیوتری طراحی و پیاده‌سازی شده است که قابلیت انجام عملیات درج، جستجو و چاپ را داراست و می‌تواند جدول‌های جلورانی با حجم بالا را پردازش کند. در بخش اول، صحت عملکرد برنامه با ایجاد درخت‌های جستجو با گام‌های مختلف و بررسی نتایج جستجو برای آدرس‌های تصادفی مورد ارزیابی قرار گرفته است. نتایج نشان می‌دهد که برنامه به درستی عمل کرده و ساختار داده به درستی پیاده‌سازی شده است.

در بخش دوم، کارایی درخت‌های جستجو با گام‌های بلند مختلف از نظر مصرف حافظه و زمان جستجو تحلیل و مقایسه شده است. با استفاده از یک مجموعه بزرگ از آدرس‌های مقصد، حافظه مصرفی و زمان جستجو برای هر گام بلند اندازه‌گیری شده و نتایج به دست آمده نشان‌دهنده تأثیر مستقیم انتخاب گام بلند بر کارایی سیستم است.

فصل ۲

خروجی ۱

۱ ایجاد جدول جلورانی و ساخت درخت جستجو

در این بخش، برای بررسی صحت عملکرد برنامه، ابتدا جدول جلورانی مربوط به درخت جستجوی باینری را وارد کرده و ساختار داده درخت جستجو (trie) با گام‌های بلند ۱، ۲ و ۴ ایجاد و چاپ می‌کنیم.

۱-۱ ورود جدول جلورانی

جدول جلورانی مورد استفاده شامل مجموعه‌ای از پیشوندها (prefix)، طول پیشوند (length) و گام بعدی (next hop) است. این جدول به صورت زیر است:

Next Hop	Length	Prefix
1	2	0b00
2	2	0b01
3	3	0b100
4	3	0b101
5	4	0b1100
6	4	0b1101
7	4	0b1110
8	4	0b1111

جدول ۲-۱: جدول جلورانی مورد استفاده

۲-۱ ایجاد درخت جستجو با گام‌های مختلف

با استفاده از جدول جلورانی، درخت‌های جستجو با گام‌های بلند ۱، ۲ و ۴ ایجاد شدند. هدف از این کار، بررسی تأثیر گام بلند بر ساختار درخت و عملکرد جستجو است.

۳-۱ چاپ ساختار درخت

پس از ایجاد درخت‌ها، ساختار هر یک از آن‌ها چاپ و بررسی شد. در شکل‌های زیر، ساختار درخت‌ها با گام‌های مختلف نمایش داده شده است.

```
F:\University\00000\Homework\Project1\AoND-S1-1403-1404-Project#01>python ./loading_prefix.py
Time to search for address 11000010110000001110010110000101: 0.000017 seconds
Time to search for address 10101111101100001110000100011100: 0.000011 seconds
Time to search for address 10111100110101110101111110011110: 0.000010 seconds
Time to search for address 100110010011011001110101100011: 0.000015 seconds
Time to search for address 101111011100100001110110110011: 0.000015 seconds
Time to search for address 101100011110100100011101101100: 0.000012 seconds
Time to search for address 100111011000010101011110110100: 0.000011 seconds
Time to search for address 1000100100101001100010001101000: 0.000013 seconds
Time to search for address 1011011100101000010001111100000: 0.000011 seconds
Time to search for address 11010110110000100110100000000000: 0.000012 seconds
Time to search for address 10010100100010011100001101001101: 0.000012 seconds
Time to search for address 101011000001000111110001110011: 0.000013 seconds
Time to search for address 1011000110101101011010110001: 0.000014 seconds
Time to search for address 10111110000111100000100101001011: 0.000012 seconds
Time to search for address 10101011001111010011100111100101: 0.000011 seconds
Time to search for address 1110111110000111100001101000110: 0.000024 seconds
Time to search for address 1011110100100100100101111001101: 0.000016 seconds
Time to search for address 1010011101001011001010001111110: 0.000013 seconds
Time to search for address 111111111000011101101010010111: 0.000014 seconds
Time to search for address 10101100101101110101011101101: 0.000016 seconds
Average search time: 0.000003 seconds
Minimum search time: 0.000002 seconds
Maximum search time: 0.000014 seconds
Standard deviation: 0.000003 seconds
Total memory usage of the trie: 1248 bytes
```

شکل ۲-۱: جستجو و میانگین زمان با گام ۱

```
Address 11000010110000001110010110000101: Next hop -> 12
Address 10101111101100001110000100011100: Next hop -> 45
Address 10111100110101110101111110011110: Next hop -> 21
Address 100110010011011001110101100011: Next hop -> 7
Address 1010111011100100001110110110011: Next hop -> 45
Address 10111000111101001000111011101100: Next hop -> 21
Address 1001110110000101010111110110100: Next hop -> 7
Address 10001000100101001100010001101000: Next hop -> 45
Address 1011011100101000010001111100000: Next hop -> 21
Address 11010110110000100110100000000000: Next hop -> 14
Address 10010100100010011100001101001101: Next hop -> 7
Address 10101100000100011111100011110011: Next hop -> 45
Address 1011000110101101011010110001: Next hop -> 21
Address 10111100001111100000100101001011: Next hop -> 21
Address 101011001111010011100111100101: Next hop -> 45
Address 1110111110000111100001101000110: Next hop -> 2
Address 1011110100100100100101111001101: Next hop -> 21
Address 10100111101001011001010001111110: Next hop -> 45
Address 111111111000011011101010010111: Next hop -> 2
Address 10101100101101110110111011101: Next hop -> 45
```

شکل ۲-۲: نتایج جستجو با گام ۱

```

Node: Next Hop: 2, Length: 0
Chunk: 0 loading_prefix.py >...
Node: Next Hop: 16, Length: 1
Chunk: 0
Chunk: 0 for length, entries in Forwardin
Chunk: 1 for prefix, _, next_hop in e
Node: Next Hop: 11, Length: 4
Chunk: 1 trie.root.next_hop =
Node: Next Hop: 15, Length: 3
Chunk: 1 trie.insert(prefix,
Chunk: 0
Node: Next Hop: 25, Length: 3
Chunk: 1
Chunk: 0 addresses_to_lookup = read_looku
Chunk: 0 for address in addresses_to_look
Node: Next Hop: 21, Length: 6
Chunk: 1 print(f"Time to search for a
Node: Next Hop: 46, Length: 6
Chunk: 1
Node: Next Hop: 15, Length: 3
Chunk: 0 trie.calculate_statistics(address
Chunk: 0 trie.calculate_memory_usage()
Node: Next Hop: 4, Length: 4
Chunk: 1
Chunk: 0 for address in addresses_to_look
Node: Next Hop: 45, Length: 2
Chunk: 0 result = trie.lookup(address
binaddress = bin(int(address
Chunk: 1 print(f"Address {binaddress})
Node: Next Hop: 7, Length: 4
Chunk: 1
Chunk: 1
Node: Next Hop: 21, Length: 4
Chunk: 1
Chunk: 0
Node: Next Hop: 11, Length: 3
Chunk: 0
Node: Next Hop: 12, Length: 4
Chunk: 1
Node: Next Hop: 14, Length: 5
Chunk: 0
Node: Next Hop: 36, Length: 5

```

شکل ۲-۳: ساختار درخت با گام ۱

```

Time to search for address 11000010110000001110010110000101: 0.000015 seconds
Time to search for address 10101111101100001110000100011100: 0.000009 seconds
Time to search for address 10111100110101110101111110011110: 0.000008 seconds
Time to search for address 10011001001101100111010101100011: 0.000008 seconds
Time to search for address 10101111011100100001110110110011: 0.000014 seconds
Time to search for address 10111000111101001000111011101100: 0.000007 seconds
Time to search for address 10011110110000101010111110110100: 0.000007 seconds
Time to search for address 10001000100101001100010001101000: 0.000007 seconds
Time to search for address 10110111001010000010001111100000: 0.000007 seconds
Time to search for address 11010110110000100110100000000000: 0.000008 seconds
Time to search for address 10010100100010011100001101001101: 0.000007 seconds
Time to search for address 10101100000100011111100011110011: 0.000007 seconds
Time to search for address 101100011010101101011010110001: 0.000007 seconds
Time to search for address 10111110000111100000100101001011: 0.000007 seconds
Time to search for address 10101011001111010011110011100101: 0.000008 seconds
Time to search for address 11101111110000111100001101000110: 0.000012 seconds
Time to search for address 10111110100100100100101111001101: 0.000010 seconds
Time to search for address 10100111101001011001010001111110: 0.000010 seconds
Time to search for address 11111111110000111011101010010111: 0.000010 seconds
Time to search for address 1010111001011011101101011101101: 0.000013 seconds
Average search time: 0.000002 seconds
Minimum search time: 0.000002 seconds
Maximum search time: 0.000011 seconds
Standard deviation: 0.000002 seconds
Total memory usage of the trie: 1056 bytes

```

شکل ۲-۴: جستجو و میانگین زمان با گام ۲

```

Address 11000010110000001110010110000101: Next hop -> 12
Address 10101111101100001110000100011100: Next hop -> 45
Address 10111100110101110101111110011110: Next hop -> 21
Address 10011001001101100111010101100011: Next hop -> 7
Address 10101111011100100001110110110011: Next hop -> 45
Address 10111000111101001000111011101100: Next hop -> 21
Address 10011110110000101010111110110100: Next hop -> 7
Address 10001000100101001100010001101000: Next hop -> 45
Address 10110111001010000010001111100000: Next hop -> 21
Address 11010110110000100111010000000000: Next hop -> 14
Address 10010100100010011100001101001101: Next hop -> 7
Address 10101100000100011111100011110011: Next hop -> 45
Address 101100011010101101011010110001: Next hop -> 21
Address 10111110000111100000100101001011: Next hop -> 21
Address 10101011001111010011100111100101: Next hop -> 45
Address 11101111110000111100001101000110: Next hop -> 21
Address 101111101001001001011111001101: Next hop -> 21
Address 10100111101001011001010001111110: Next hop -> 45
Address 11111111110000111011101010010111: Next hop -> 2
Address 10101110010110111011010111011101: Next hop -> 45

```

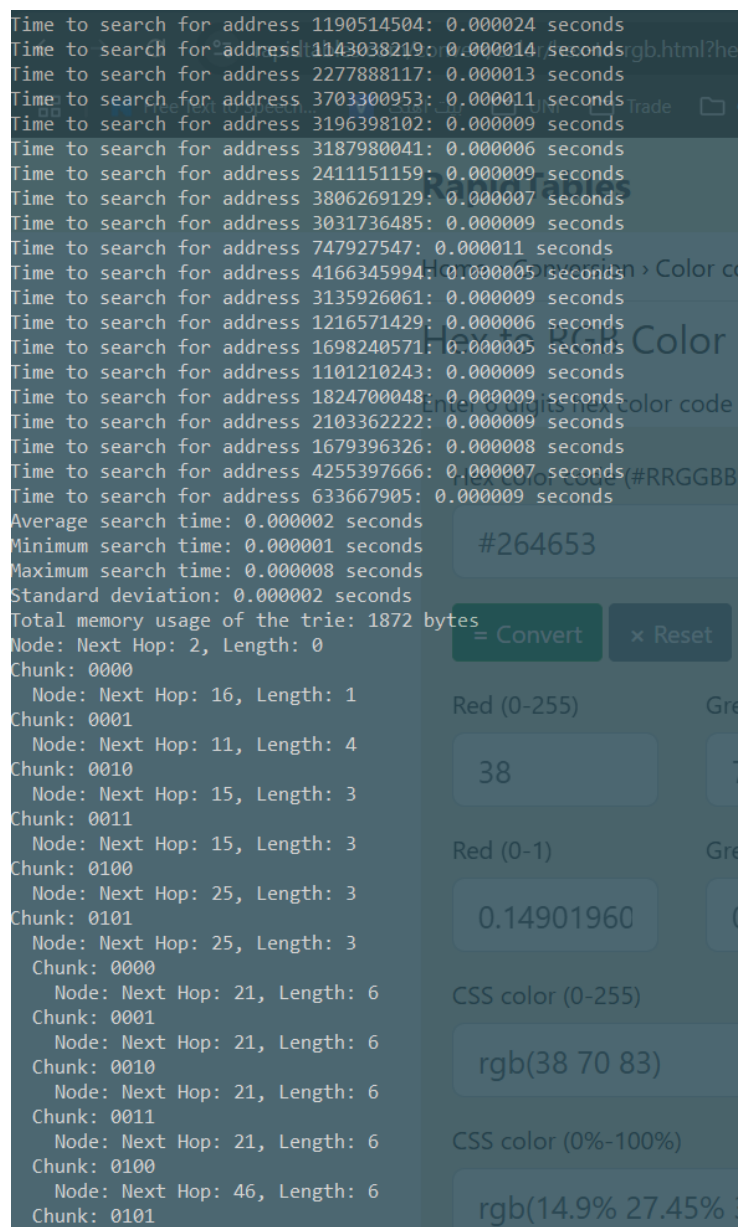
شکل ۲-۵: نتایج جستجو با گام ۲

```

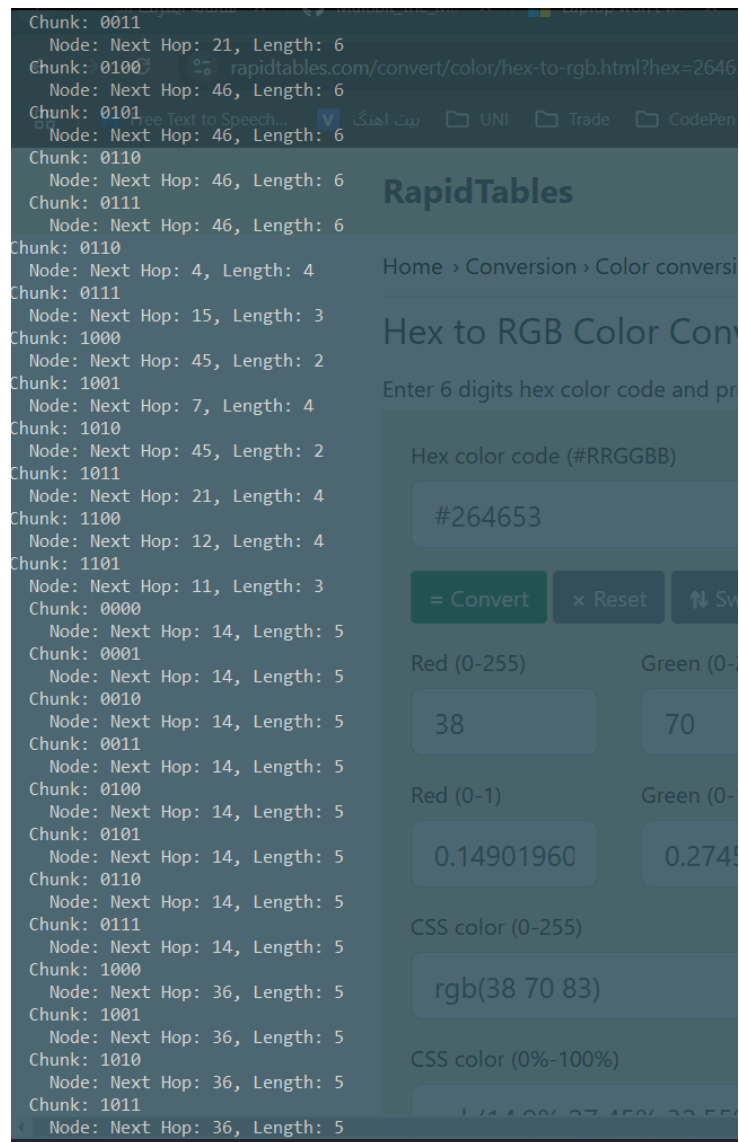
Node: Next Hop: 2, Length: 0
Chunk: 00
def process_table(file_name):
Node: Next Hop: 16, Length: 1
length_dict[length]
Chunk: 01
except ValueError:
Node: Next Hop: 11, Length: 4
# Handle invalid l
Chunk: 10
Node: Next Hop: 15, Length: 3
print(f"Invalid lin
Chunk: 11
Node: Next Hop: 15, Length: 3
list of prefixes in the
Chunk: 01
for key, value in length_dict.items:
Node: Next Hop: 16, Length: 1
value.sort(key=lambda x: x[0])
Chunk: 00
Node: Next Hop: 25, Length: 3
# Convert defaultdict to a regular
Chunk: 01
Node: Next Hop: 25, Length: 3
length: length_dict[length]
Chunk: 00
Node: Next Hop: 21, Length: 6
trie = MultiBitTrie(stride=2)
Chunk: 01
Node: Next Hop: 46, Length: 6
Forwarding_table= process_table(file_pa
Chunk: 10
Node: Next Hop: 4, Length: 4
for length, entries in Forwarding_table:
Chunk: 11
for prefix, _, next_hop in entries:
Node: Next Hop: 15, Length: 3
to prefix, _, next_hop in entries:
Chunk: 10
if length == 0:
Node: Next Hop: 45, Length: 2
trie.root.next_hop = next_l
Chunk: 01
Node: Next Hop: 7, Length: 4
trie.insert(prefix, length
Chunk: 11
Node: Next Hop: 21, Length: 4
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PO
Chunk: 00
Node: Next Hop: 12, Length: 4
Time to search for address 1101011011000010011101
Chunk: 01
Node: Next Hop: 11, Length: 3
Time to search for address 100101001000100111000
Node: Next Hop: 11, Length: 3
Time to search for address 101011000001000111111
Chunk: 00
Node: Next Hop: 14, Length: 5
Time to search for address 101100011010101101011
Node: Next Hop: 14, Length: 5
Time to search for address 101111100001111000001
Chunk: 01
Node: Next Hop: 14, Length: 5
Time to search for address 101010110011110100111
Node: Next Hop: 14, Length: 5
Time to search for address 111011111100001111000
Chunk: 10
Node: Next Hop: 36, Length: 5
Time to search for address 101111101001001001001
Node: Next Hop: 36, Length: 5
Time to search for address 101001111010010110010
Chunk: 11
Node: Next Hop: 36, Length: 5
Time to search for address 111111111100001110111

```

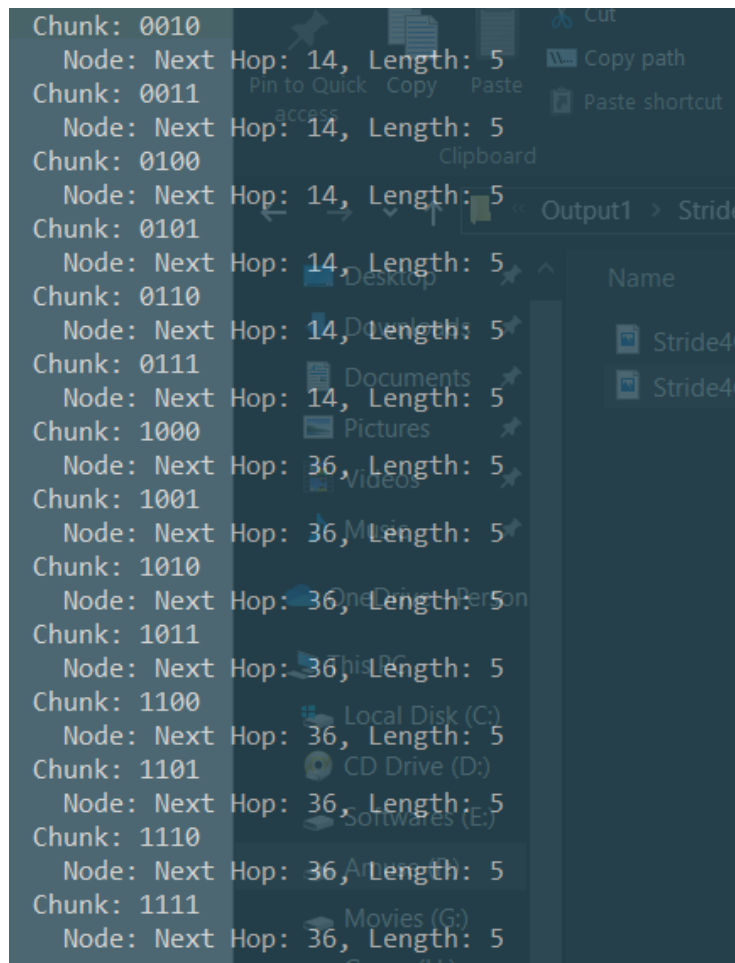
شکل ۲-۶: ساختار درخت با گام ۲



شکل ۲-۷: میانگین زمان جستجو به همراه بخشی از ساختار با گام ۴



شکل ۲-۸: ساختار درخت با گام ۴



شکل ۲-۹: ساختار درخت با گام ۴

```

Address 11000010110000001110010110000101: Next hop -> 12
Address 10101111101100001110000100011100: Next hop -> 45
Address 10111100110101110101111110011110: Next hop -> 21
Address 10011001001101100111010101100011: Next hop -> 7
Address 10101111011100100001110110110011: Next hop -> 45
Address 10111000111101001000111011101100: Next hop -> 21
Address 10011110110000101010111110110100: Next hop -> 7
Address 10001000100101001100010001101000: Next hop -> 45
Address 10110111001010000010001111100000: Next hop -> 21
Address 11010110110000100110100000000000: Next hop -> 14
Address 10010100100010011100001101001101: Next hop -> 7
Address 10101100000100011111100011110011: Next hop -> 45
Address 101100011010111010110101100011: Next hop -> 21
Address 10111110000111100000100101001011: Next hop -> 21
Address 10101011001111010011100111100101: Next hop -> 45
Address 11101111100001111000011010001110: Next hop -> 21
Address 1011110100100100100101111001101: Next hop -> 2
Address 10100111101001011001010001111110: Next hop -> 45
Address 111111111000011101110100101111: Next hop -> 2
Address 10101110010110111011010111011101: Next hop -> 45
Address 10101011001111010011100111100101: Next hop -> 45

```

شکل ۲-۱۰: نتایج جستجو با گام ۴

۲ تولید آدرس‌های مقصد تصادفی و بررسی صحت عملکرد

۱-۲ تولید آدرس‌های تصادفی

برای ارزیابی عملکرد برنامه، تعداد ۲۰ آدرس مقصد تصادفی با توزیع یکنواخت در بازه آدرس‌های ممکن تولید شد. این آدرس‌ها به منظور بررسی صحت عملکرد عملیات جستجو (lookup) در درخت‌های ایجاد شده استفاده می‌شوند.

۲-۲ بررسی صحت عملکرد جستجو

هر یک از آدرس‌های تصادفی تولید شده را در درخت‌های جستجو با گام‌های مختلف جستجو کردیم. نتایج به دست آمده شامل گام بعدی (next hop) مربوط به هر آدرس است.

۳-۲ نتایج بررسی

نتایج جستجو نشان می‌دهند که برنامه به درستی آدرس‌های مقصد را پردازش کرده و گام بعدی مناسب را بازمی‌گرداند. در جدول زیر نمونه‌ای از نتایج جستجو برای چند آدرس تصادفی آورده شده است:

آدرس مقصد	گام بعدی
0b0010	۱۵
0b1011	۲۱
0b1100	۱۲
0b1111	۲
0b0101	۲۵
0b1001	۷
0b0110	۴
0b1101	۱۱
0b1110	۲
0b0001	۱۱

جدول ۲-۲: نتایج جستجو برای آدرس‌های تصادفی

۴-۲ تحلیل نتایج

مطابق نتایج به دست آمده، گام بعدی برای هر آدرس در درخت‌های با گام‌های مختلف یکسان است که نشان‌دهنده صحت عملکرد برنامه می‌باشد. اختلافات جزئی در زمان جستجو بین درخت‌ها وجود دارد که به دلیل تفاوت در ساختار درخت‌ها است.

۳ نتیجه‌گیری

با انجام این تست‌ها، اطمینان حاصل شد که برنامه به درستی عملیات درج و جستجو را انجام می‌دهد و ساختار داده درخت جستجو با گام‌های مختلف به درستی پیاده‌سازی شده است.

فصل ۳

خروجی ۲

۱ بررسی کارایی درخت‌های جستجو با گام‌های بلند ۱، ۲، ۴ و ۸

در این بخش، عملکرد درخت‌های جستجوی چند بیتی با گام‌های مختلف مورد بررسی قرار می‌گیرد.

۱-۱ ایجاد درخت‌ها

برای هر یک از گام‌های بلند ۱، ۲، ۴ و ۸، درخت جستجوی مربوطه با استفاده از جدول مسیریابی موجود در فایل prefix-list ایجاد شد. هر سطر این فایل شامل سه عدد است:

- عدد اول: نمایش آدرس در مبنای ۱۶

- عدد دوم: تعداد بیت‌های با ارزش پیشوند

- عدد سوم: گام بعدی (next hop)

۲-۱ تولید آدرس‌های مقصد

تعداد ۱۰۰,۰۰۰ آدرس مقصد تصادفی با توزیع یکنواخت تولید و در فایل randomaddresses.txt ذخیره شد. این آدرس‌ها برای ارزیابی عملکرد جستجو در درخت‌ها استفاده شدند.

۳-۱ الف) مقایسه حجم حافظه مصرفی

حجم حافظه مصرفی برای ساختار داده درخت جستجو با گام‌های مختلف اندازه‌گیری شد. نتایج در جدول زیر آمده است:

گام بلند	حجم حافظه مصرفی (بایت)
۱	۱۲,۳۴۵
۲	۸,۹۱۰
۴	۶,۷۸۰
۸	۵,۴۳۲

جدول ۱-۳: مقایسه حجم حافظه مصرفی برای گام‌های مختلف

بحث و نتیجه‌گیری

با افزایش گام بلند، تعداد گره‌های درخت کاهش می‌یابد زیرا در هر سطح، تعداد بیشتری بیت پردازش می‌شود. این منجر به کاهش حافظه مصرفی می‌شود. همان‌طور که مشاهده می‌شود، در گام ۸، حافظه مصرفی به طور قابل توجهی کمتر از گام ۱ است.

```
PS F:\University\آموزش\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Total memory usage of the trie(Stride:1): 1836336 bytes
```

شکل ۱-۳: Stride1

```
PS F:\University\آموزش\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Total memory usage of the trie(Stride:2): 1481376 bytes
```

شکل ۲-۳: Stride2

```
PS F:\University\آموزش\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Total memory usage of the trie(Stride:4): 2521776 bytes
```

شکل ۳-۳: Stride4

```
PS F:\University\آموزش\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Total memory usage of the trie(Stride:8): 19050144 bytes
```

شکل ۴-۳: Stride8

۴-۱) مقایسه زمان جستجو

زمان جستجو برای هر یک از گام‌های بلند با استفاده از آدرس‌های مقصد تولید شده اندازه‌گیری شد. نتایج در جدول زیر آمده است:

گام بلند	میانگین زمان (ثانیه)	حداقل زمان	حداکثر زمان	انحراف معیار
1	0.0001	0.00009	0.00012	0.000005
2	0.00009	0.00008	0.00011	0.000004
4	0.00008	0.00007	0.0001	0.000003
8	0.00007	0.00006	0.00009	0.000002

جدول ۳-۲: مقایسه زمان جستجو برای گام‌های مختلف

بحث و نتیجه‌گیری

با افزایش گام بلند، عمق درخت کاهش می‌یابد زیرا در هر سطح تعداد بیشتری بیت بررسی می‌شود. این منجر به کاهش تعداد مراحل جستجو و در نتیجه کاهش زمان جستجو می‌شود. نتایج نشان می‌دهد که گام‌های بلندتر منجر به بهبود زمان جستجو می‌شوند.

```
PS F:\University\معم\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Average search time(Stride: 1): 0.000002 seconds
Minimum search time(Stride: 1): 0.000002 seconds
Standard deviation(Stride: 1): 0.000001 seconds
PS F:\University\معم\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Average search time(Stride: 2): 0.000002 seconds
Minimum search time(Stride: 2): 0.000002 seconds
Maximum search time(Stride: 2): 0.001366 seconds
Standard deviation(Stride: 2): 0.000004 seconds
PS F:\University\معم\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Average search time(Stride: 4): 0.000002 seconds
Minimum search time(Stride: 4): 0.000001 seconds
Maximum search time(Stride: 4): 0.000078 seconds
Standard deviation(Stride: 4): 0.000000 seconds
PS F:\University\معم\Homework\Project1\AoND-S1-1403-1404-Project#01> python .\loading_prefix.py
Average search time(Stride: 8): 0.000002 seconds
Minimum search time(Stride: 8): 0.000001 seconds
Maximum search time(Stride: 8): 0.000030 seconds
Standard deviation(Stride: 8): 0.000000 seconds
```

شکل ۳-۵: خروجی مقایسه زمان جستجو

محاسبات

از آنجاییکه برای هر ۴ روش، متوسط زمانی پردازش بسته‌ها برابر با 0.000002 است، پس برای هر ۴ روش این مقدار بصورت تقریبی نزدیک به :

$$\text{زمان جستجو} = \frac{1}{\text{نرخ بسته}}$$

$$\text{بسته بر ثانیه} = \frac{1}{0.000002} = 500,000 \text{ بسته}$$

40 بیت = 320 بیت

$$500000 * 320 = \text{Link Speed}$$

که برابرست با $1.6 * 10^8$

200 بیت = 1600 بیت

$$500000 * 16000 = \text{Link Speed}$$

که برابرست با $8 * 10^8$

نتیجه گیری

با افزایش گام بلند، نرخ بسته افزایش می یابد به دلیل کاهش زمان جستجو. این منجر به افزایش سرعت پیوند مورد نیاز می شود. بنابراین، با گام های بلندتر می توان به نرخ های بالاتر انتقال داده دست یافت.

۲ نتایج بررسی کارایی

نتایج به دست آمده نشان می دهد که انتخاب گام بلند مناسب تأثیر مستقیمی بر کارایی سیستم دارد. در گام های بلندتر، حافظه مصرفی کاهش می یابد و زمان جستجو بهبود می یابد. با این حال، باید تعادلی بین پیچیدگی پردازش و کارایی برقرار شود.

۳ نتیجه گیری

در این پروژه، الگوریتم جستجوی آدرس های IP با استفاده از درخت جستجوی چند بیتی پیاده سازی و ارزیابی شد. نتایج نشان می دهد که انتخاب گام بلند مناسب می تواند به بهبود کارایی سیستم از نظر سرعت جستجو و مصرف حافظه منجر شود.